

DBMS Project – Week 1

Project Title: E-Commerce Order Management System for a Pet Supplies Store

Requirement Analysis

Understanding the E-Commerce Pet Supplies Store System

A Pet Supplies Store E-Commerce Order Management System is an online platform that allows customers to purchase pet food, toys, accessories, grooming products, and healthcare items. The system requires a database to maintain important information about customers, products, orders, payments, and inventory. Properly organized data helps the store manage customer activities, product details, and transactions efficiently.

The Process Flow

- **Customer Registration:** Customers create an account by providing their personal details.
- **Product Search:** Customers browse and search for available pet supplies.
- **Cart Management:** Customers add their selected products to the shopping cart.
- **Order Confirmation:** Customers provide delivery details and confirm their orders.
- **Payment:** Customers select a payment method and complete the payment process.
- **Product Delivery:** The store processes the order and delivers the products to the customer's address.

- Feedback and Assistance: Customers can provide feedback or contact support regarding their orders.

Entities

Entity	Description	Key Attributes
Customer	Stores information about customers who purchase pet supplies.	Customer_id (PK), first_name, last_name, email, phone, created_at
Product	Contains details of pet supplies available for purchase.	Product_id (PK), product_name, category, price, stock_quantity, description
Order	Records purchases made by customers through the platform.	Order_id (PK), customer_id (FK), order_date, status, total_amount, shipping_address
Order_Detail	Stores information about the products included in each order.	Order_detail_id (PK), order_id (FK), product_id (FK), quantity, price
Payment	Maintains payment information related to customer orders.	Payment_id (PK), order_id (FK), payment_method, status, transaction_id, amount

Inventory	Maintains the available stock of pet supplies.	Inventory_id (PK), product_id (FK), stock_quantity, last_updated
Delivery	Stores delivery details and order tracking information.	Delivery_id (PK), order_id (FK), delivery_date, delivery_status, tracking_number

Key Relationships

- Customer has a One-to-Many relationship with Order.
- Order has a One-to-Many relationship with Order_Detail.
- Product has a One-to-Many relationship with Order_Detail.
- Order has a One-to-One relationship with Payment.
- Product has a One-to-One relationship with Inventory.
- Order has a One-to-One relationship with Delivery.

Customer Table Creation

The Customer table is created to maintain the basic details of customers registered on the e-commerce platform. A unique Customer_id is used to identify each customer record.

```
SQL> CREATE TABLE Customer (Customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50), email VARCHAR(100) UNIQUE, phone VARCHAR(15), created_at DATE);
```

```
Table created.
```

Describe Customer Table

After creating the Customer table, the DESCRIBE command is used to check the structure of the table. It displays the column names, data types, and other details associated with each attribute.

```
SQL> DESC Customer;
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	NUMBER(38)
FIRST_NAME	NOT NULL	VARCHAR2(50)
LAST_NAME		VARCHAR2(50)
EMAIL		VARCHAR2(100)
PHONE		VARCHAR2(15)
CREATED_AT		DATE

Insert Values

The INSERT command is used to add customer information into the Customer table. Multiple records can be entered based on the number of customers required for the database.

```
SQL> INSERT INTO Customer VALUES (1, 'nivedha', 'senthil kumar', 'nivedha@gmail.com', '8990555523', TO_DATE('2026-05-04', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
SQL> INSERT INTO Customer VALUES (2, 'Arun', 'Kumar', 'arun@gmail.com', '9876543210', TO_DATE('2026-05-05', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
SQL> INSERT INTO Customer VALUES (3, 'Priya', 'Raj', 'priya@gmail.com', '9876543211', TO_DATE('2026-05-06', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
SQL> INSERT INTO Customer VALUES (4, 'Rahul', 'Sharma', 'rahul@gmail.com', '9876543212', TO_DATE('2026-05-07', 'YYYY-MM-DD'));
```

```
1 row created.
```

Display Customer Records

The stored customer information can be viewed using the SELECT command.

```
SQL> SELECT * FROM Customer;
```

```
CUSTOMER_ID FIRST_NAME
```

```
LAST_NAME
```

```
EMAIL
```

```
PHONE CREATED_A
```

```
1 nivedha  
senthil kumar  
nivedha@gmail.com  
8990555523 04-MAY-26
```

```
CUSTOMER_ID FIRST_NAME
```

```
LAST_NAME
```

```
EMAIL
```

```
PHONE CREATED_A
```

```
2 Arun  
Kumar  
arun@gmail.com  
9876543210 05-MAY-26
```

```
CUSTOMER_ID FIRST_NAME
```

```
LAST_NAME
```

```
EMAIL
```

```
PHONE CREATED_A
```

```
3 Priya  
Raj  
priya@gmail.com  
9876543211 06-MAY-26
```

```
CUSTOMER_ID FIRST_NAME
```

```
LAST_NAME
```

```
EMAIL
```

```
PHONE CREATED_A
```

```
4 Rahul  
Sharma  
rahul@gmail.com  
9876543212 07-MAY-26
```

update

The UPDATE command is used to change existing customer information in the Customer table.

```
SQL> UPDATE Customer set PHONE = 9867356421 WHERE CUSTOMER_ID = 1;  
1 row updated.
```

NIVEDHA.S

BSC CS WITH AI [S-1]

E25AI131